

● 오라클 9일차

- 테이블 복제, CTAS, 제약조건 중에는 not null 만 복제됨
 - show user
 - SELECT * FROM tab;
 - DROP TABLE hr.emp PURGE;
 - SELECT * FROM hr.employees;
 - -- 내가 받은 시스템권한 (sys 유저로부터 직접받은 권한)
 - SELECT * FROM user_sys_privs;
 -
 - -- ROLE(롤) : 유저에 부여할 수 있는 권한을 모아 놓은 객체, 유저 관리에 대한 편리성
 - SELECT * FROM session_roles;
 - -- 롤안에 시스템 권한 확인
 - SELECT * FROM dba_sys_privs WHERE grantee = 'CONNECT';
 - SELECT * FROM dba_sys_privs WHERE grantee = 'RESOURCE';
 - -- 내가 받은 롤안에 시스템 권한 확인
 - SELECT * FROM role_sys_privs;
 - CREATE TABLE hr.emp
 - TABLESPACE ;
 - -- 유저한테 부여한 롤 확인
 - SELECT * FROM dba_role_privs WHERE grantee = 'HR';
 - -- 유저한테 직접 부여한 시스템 권한 확인
 - SELECT * FROM dba_sys_privs WHERE grantee = 'HR';
 - SELECT * FROM user_sys_privs;
 - SELECT * FROM role_sys_privs;
 - SELECT * FROM user_tab_privs;
 - SELECT * FROM hr.employees;
 - SELECT * FROM hr.locations;
 - -- hr.locations 테이블에 대한 SELECT 권한을 insa 유저한테 부여
 - GRANT SELECT ON hr.locations TO insa;
 - SELECT * FROM dba_tab_privs WHERE grantee = 'INSA';
 - -- hr.locations 테이블에 대한 SELECT 권한을 insa 유저한테 회수
 - REVOKE SELECT ON hr.loations FROM insa;

- REVOKE SELECT ON hr.employees FROM insa;
- REVOKE SELECT ON hr.departments FROM insa;

- -- 직접 받은 시스템 권한과 롤로 통해 받은 시스템 권한 한번에 확인방법
- SELECT * FROM session_privs;

- -- 테이블 생성하려면 두가지 체크
- /*
- 1) CREATE TABLE 시스템 권한
- 2) 테이블스페이스에 대한 사용 할 수 있는지 체크 (quota) */

- CREATE TABLE hr.emp
- TABLESPACE users
- AS
- SELECT employee_id, last_name || ' ' || first_name name, salary*12 sal, department_id
- FROM hr.employees;
- -- WHERE 1= 2; -- 조건절을 false로 구성하게 되면 테이블의 구조만 복제되고 데이터는 복제가 안됨

- SELECT * FROM hr.emp;

- DROP TABLE hr.emp PURGE;

- CREATE TABLE hr.emp
- TABLESPACE users
- AS
- SELECT *
- FROM hr.employees
- WHERE 1= 2;

- -- INSERT subquery
 - -- 삽입하고자 하는 데이터를 받을 테이블에 형태가 같지 않을 경우 오류 발생
 - INSERT INTO hr.emp
 - SELECT * FROM hr.employees;

 - /* 문제 hr.employees 테이블에서 관리자 사원들의 employee_id, last_name, hire_date를
 - hr.mgr테이블에 데이터를 로드해라 */

 - -- 관리자 정보를 추출하지 않고 한 경우
 - INSERT INTO hr.mgr
 - SELECT employee_id, last_name, hire_date FROM hr.employees;

 - SELECT * FROM hr.mgr;

 - DROP TABLE hr.mgr PURGE;

- CREATE TABLE hr.mgr(id number(3), name varchar2(30), day date) TABLESPACE users;
- – 관리자 정보를 추출해서 한 경우
- INSERT INTO hr.mgr(id,name,day)
- SELECT employee_id, last_name, hire_date
- FROM hr.employees o
- WHERE EXISTS (SELECT 'x'
- FROM hr.employees
- WHERE manager_id = o.employee_id);
- SELECT * FROM hr.mgr;
- COMMIT;
- 받고자 하는 데이터를 추출할 때는 서브쿼리를 이용해서 원하는 데이터를 따로 추출한 후 INSERT를 해줘야 된다 즉 SELECT문을 잘 작성해야지 된다.
- -- 자동 COMMIT 발생하는 경우
- -- DDL (CREATE, ALTER, DROP, RENAME, TRUNCATE, COMMENT)
- -- DCL (GRANT, REVOKE)
- -- SQL*PLUS에서 EXIT를 수행해서 종료할 경우
- -- SQL*PLUS에서 CONNECT(CONN)을 이용해서 다른 유저로 접속한 경우
- -- 자동 ROLLBACK이 발생하는 경우
- -- SQL*PLUS를 비정상적인 종료를 할 경우(창닫기)
- -- DML 작업을 수행하고 있는 컴퓨터가 비정상적으로 종료
- -- CLIENT - SERVER 환경에서 네트워크가 장애가 발생하는 경우
- -- SAVEPOINT
- -- DML 작업시에 롤백을 도와주는 작업
- -- SAVEPOINT 표시자이름;
- -- ROLLBACK TO SAVEPOINT 표시자이름; (취소)
- DELETE FROM hr.emp WHERE dept_id = 90; -- transaction 시작 (진행중)
- CREATE TABLE hr.dept
- AS SELECT * FROM hr.departments;
- SELECT * FROM hr.dept;
- SELECT * FROM hr.emp WHERE dept_id = 90;
- ROLLBACK;
- -- DDL,DCL 문장은 자동 COMMIT이 진행되고 있어 의미 없는 롤백이 적용
- CREATE TABLE hr.new
- SELECT * FROM hr.departments; -- 문법 오류 발생
- SELECT * FROM hr.emp WHERE dept_id = 50;

- ROLLBACK;
- SELECT * FROM hr.emp WHERE dept_id = 50;
- CREATE TABLE hr.test(id number) TABLESPACE users;
- SELECT * FROM hr.test;
- INSERT INTO hr.test(id) VALUES(1); -- transaction 시작
- SELECT * FROM hr.test;
- SAVEPOINT t1;
- INSERT INTO hr.test(id) VALUES(2);
- SELECT * FROM hr.test;
- SAVEPOINT t2;
- INSERT INTO hr.test(id) VALUES(300);
- SELECT * FROM hr.test;
- ROLLBACK; -- transaction이 시작된 곳까지 전부 취소
- ROLLBACK TO t1;
- ROLLBACK TO t2;
- SELECT * FROM hr.test; -- 그로 인해 데이터 확인시 데이터 값이 없음
- CREATE TABLE hr.sal_history (id number, day date, sal number)
- TABLESPACE users;
-
- CREATE TABLE hr.mgr_history(id number, mgr number, sal number)
- TABLESPACE users;
-
- INSERT INTO hr.sal_history(id,day,sal)
- SELECT employee_id, hire_date, salary
- FROM hr.employees;
-
- INSERT INTO hr.mgr_history(id,mgr,sal)
- SELECT employee_id, manager_id, salary
- FROM hr.employees;
-
- SELECT * FROM hr.sal_history;

- SELECT * FROM hr.mgr_history;
- -- 다중테이블 INSERT(9i)
 - -- 소스 테이블에서 데이터를 추출해서 여러 타겟 테이블에 데이터를 로드 (적재, insert)하는 SQL문
 - -- ETL(Extraction(추출),Transformation(변형),Loading(적재))
 -
 - -- 무조건 INSERT ALL
 -
 - INSERT ALL
 - INTO hr.sal_history(id,day,sal) values(employee_id, hire_date, salary)
 - INTO hr.mgr_history(id,mgr,sal) values (employee_id, manager_id, salary)
 - SELECT employee_id, hire_date, salary
 - FROM hr.employees;
 -
 - SELECT * FROM hr.sal_history;
 - SELECT * FROM hr.mgr_history;
 -
 - ROLLBACK;
 -
 - CREATE TABLE hr.emp_history (id number, day date, sal number)
 - TABLESPACE users;
 -
 - CREATE TABLE hr.emp_sal(id number, comm number, sal number)
 - TABLESPACE users;
 -
 - -- 조건 INSERT ALL
 -
 - INSERT ALL
 - WHEN day < TO_DATE('2005-01-01','YYYY-MM-DD') AND sal >= 5000 THEN
 - INTO hr.emp_history(id,day,sal) values(id,day,sal)
 - WHEN comm IS NOT NULL THEN
 - INTO hr.emp_sal (id,comm,sal) values(id,comm,sal)
 - SELECT employee_id id, hire_date day, salary sal, commission_pct comm
 - FROM hr.employees;
 -
 - SELECT employee_id id, hire_date day, salary sal, commission_pct comm
 - FROM hr.employees
 - WHERE hire_date < TO_DATE('2005-01-01','YYYY-MM-DD') AND salary >= 5000 ;
 -
 - SELECT * FROM hr.emp_history;
 - SELECT * FROM hr.emp_sal;
 -
 - SELECT *
 - FROM hr.emp_history o
 - WHERE EXISTS(SELECT 'a'

- FROM hr.emp_sal
- WHERE id = o.id);
- -- 조건 FIRST INSERT

- CREATE TABLE hr.sal_low
- (id number, name varchar2(30), sal number)
- TABLESPACE users;
-
- CREATE TABLE hr.sal_mid
- (id number, name varchar2(30), sal number)
- TABLESPACE users;

- CREATE TABLE hr.sal_high
- (id number, name varchar2(30), sal number)
- TABLESPACE users;

- SELECT * FROM hr.sal_low;
- SELECT * FROM hr.sal_mid;
- SELECT * FROM hr.sal_high;

- -- 기존에 했던 방식

- INSERT INTO hr.sal_low(id,name,sal)
- SELECT employee_id, salary, last_name
- FROM hr.employees
- WHERE salary < 5000;

- INSERT INTO hr.sal_mid(id,name,sal)
- SELECT employee_id, salary, last_name
- FROM hr.employees
- WHERE salary BETWEEN 5000 AND 10000;

- INSERT INTO hr.sal_high(id,name,sal)
- SELECT employee_id, salary, last_name
- FROM hr.employees
- WHERE salary > 10000;

- INSERT FIRST
- WHEN salary < 5000 THEN
- INTO hr.sal_low (id,name,sal) values (employee_id,last_name,salary)
- WHEN salary BETWEEN 5000 AND 10000 THEN
- INTO hr.sal_mid (id,name,sal) values (employee_id,last_name,salary)
- WHEN salary > 10000 THEN
- INTO hr.sal_high (id,name,sal) values (employee_id,last_name,salary)
- SELECT employee_id, salary, last_name
- FROM hr.employees;

- SELECT * FROM hr.sal_low;

- SELECT * FROM hr.sal_mid;
- SELECT * FROM hr.sal_high;
- ROLLBACK;